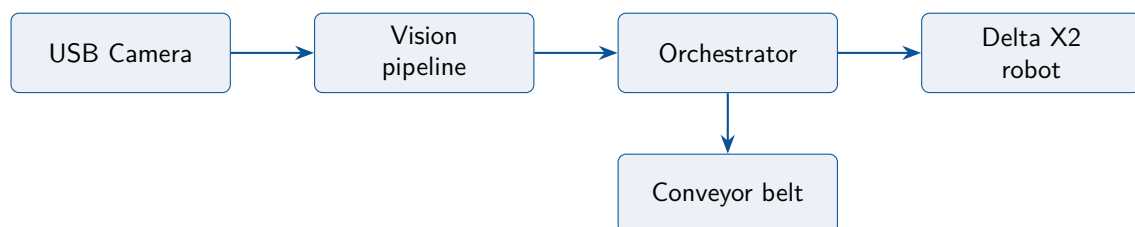


Delta X2 LEGO Sorting System

Operations Manual

Installation ■ Configuration ■ Operation



Contents

1	System Overview	1
1.1	Purpose	1
1.2	Hardware components	1
1.3	Software architecture	1
1.4	Safety design	2
2	Installation	3
2.1	Prerequisites	3
2.1.1	Hardware	3
2.1.2	Software	3
2.2	Supported platform	3
2.3	System packages	3
2.4	Rust toolchain	4
2.5	Serial port permissions	4
2.6	Building and first run	4
2.7	Troubleshooting	4
3	Configuration	7
3.1	The configuration file	7
3.2	Section [robot]	7
3.3	Section [conveyor]	7
3.4	Section [camera]	7
3.5	Section [sorting]	8
3.6	Section [vision]	8
3.7	Camera calibration	8
3.8	Complete example	8
4	Operation	11
4.1	Command line	11
4.2	Startup sequence	11
4.3	Operator interface	11
4.4	Emergency stop and recovery	11
4.5	Routine operation checklist	12
5	Maintenance and Development	13
5.1	Project layout	13
5.2	Tests	13
5.3	Development workflow	13
5.4	Editing this manual	13
5.5	Remaining work	13
A	Delta X2 G-code Protocol Reference	15
A.1	Communication protocol	15
A.2	Identification and status	15
A.2.1	IsDelta	15
A.2.2	DeltaState	15

A.2.3	Position	15
A.3	Movement commands	16
A.3.1	G00 / G01 — linear movement	16
A.3.2	G28 — homing	16
A.3.3	G90 / G91 — positioning mode	16
A.4	Synchronisation — FEEDBACK	16
A.5	End effector controls	17
A.6	Physical workspace (SP-X2)	17
A.7	Quick reference	17
Index		19

1 System Overview

1.1 Purpose

The Delta X2 LEGO Sorting System is a Rust application that drives a Delta X2 delta robot to pick LEGO bricks from a moving conveyor belt and sort them into bins. A USB camera watches the belt upstream of the robot; a vision pipeline detects bricks, converts their pixel positions into robot coordinates, and an orchestrator schedules pick-and-place moves that are executed as G-code over a serial connection.

1.2 Hardware components

Component	Interface and protocol
Delta X2 robot (SP-X2)	USB serial (<code>/dev/ttyUSB0</code>), G-code at 115 200 baud. Workspace: $X/Y \in [-160, 160]$ mm, $Z \in [-200, 0]$ mm ($Z=0$ at the top). Full protocol: Appendix A.
Conveyor belt	Separate USB serial (<code>/dev/ttyUSB1</code>); M3 S<speed> to start, M5 to stop.
USB camera	UVC device via OpenCV; default 1280×720. Fixed, even lighting over the belt is required for reliable detection.
Gripper / suction	Driven through the robot controller: M03 on, M05 off.

Table 1.1: Hardware interfaces

1.3 Software architecture

The application is asynchronous (tokio) with a Slint GUI. It is organised in three layers plus the UI:

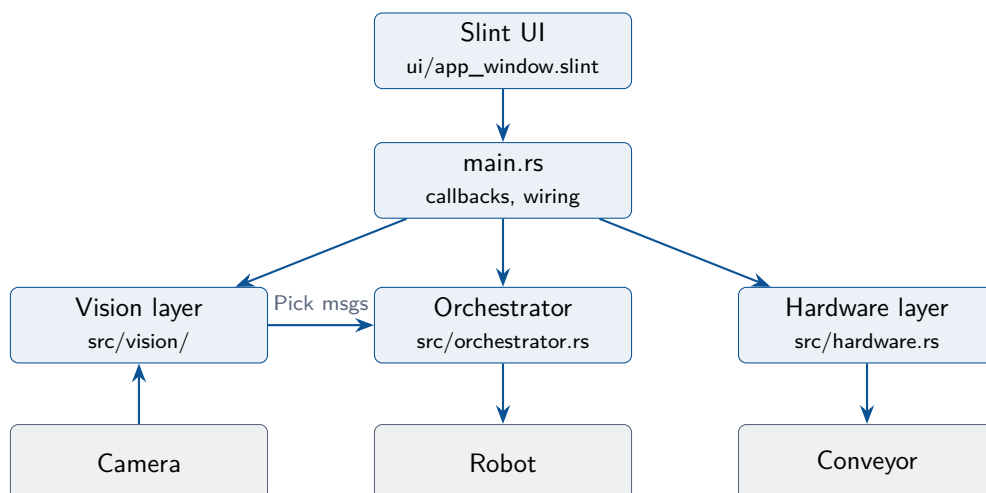


Figure 1.1: Layered architecture. Hardware access always goes through the trait abstractions of the hardware layer, so every component can also run against mock implementations (`-mock`).

Hardware layer (src/hardware.rs)

Async traits `RobotController`, `ConveyorController` and `CameraDriver`, each with a real and a mock implementation. The real robot driver validates every target position against the configured workspace before any G-code is sent, and synchronises on the robot's `FEEDBACK` echo so a command completes only when the physical move has finished.

Vision layer (src/vision/)

Blob detection (grayscale, blur, threshold, contours), pixel-to-world calibration (affine transform with rotation), an object tracker and a classifier. Tracker and classifier are still placeholder implementations (see the project TODO list).

Orchestrator (src/orchestrator.rs)

Receives messages (pick requests, raw commands, pause/resume, emergency stop) over a channel, maintains the instruction queue and executes commands one at a time on the robot.

UI (ui/app_window.slint)

Operator dashboard with start/pause, conveyor stop, homing and a prominent emergency-stop button.

1.4 Safety design

Three mechanisms protect the machine and its surroundings:

1. **Configuration validation at startup.** `Settings.toml` is checked before any hardware is touched; a pick height outside the robot's physical Z range, or a drop bin outside the workspace, aborts startup with a clear error.
2. **Workspace enforcement per move.** Every `move_to` target is validated against the configured limits inside the robot driver itself. An out-of-bounds target is rejected; it can never reach the hardware.
3. **Preemptive emergency stop.** The E-stop button writes the halt commands (M112 to the robot, M5 to the conveyor) on *dedicated serial handles* that bypass the normal command queue and locks, so the halt is delivered even while a move is in flight. The orchestrator queue is cleared and paused at the same time.

Safety warning

The software E-stop depends on a working USB serial link and on the robot firmware processing M112. It is *not* a substitute for a hardware emergency stop on the robot's power circuit. Always keep the physical E-stop within reach during operation.

2 Installation

2.1 Prerequisites

The system is designed around the following hardware and software.

2.1.1 Hardware

- **Delta X2 delta robot** (basic kit, docs.deltaxrobot.com), connected over USB serial.
- **Conveyor belt** with a G-code speed controller, connected over USB serial.
- **USB camera** mounted above the belt. The exact model is not chosen yet; all capture parameters (resolution, frame rate, pixel format) are configurable in the `[camera]` section (default 1280×720 @ 30 fps).
- **Target computer: Raspberry Pi 4 or 5** running a 64-bit Linux (ARM64/aarch64).
- **Operator display: official Raspberry Pi 7" Touch Display** (800×480 , DSI, capacitive touch). The user interface is designed touch-first for this resolution: finger-sized controls, no scrolling on the main operator view, and the emergency-stop button always remains the largest target on screen.

2.1.2 Software

- Linux only (see *Supported platform* below); no Windows or macOS support is intended.
- Rust toolchain ≥ 1.85 (the crate uses the 2024 edition).
- The system packages listed in the next section (OpenCV, libclang, libudev).

Development and testing do not require the robot hardware: an x86_64 Ubuntu machine can run the full application with simulated hardware (`cargo run -- --mock`). The production build path for the Raspberry Pi (cross-compilation from the development machine versus building on the Pi itself) is not yet fixed; until it is, the repository deliberately carries no deployment tooling.

2.2 Supported platform

Linux is the supported platform; Ubuntu 24.04 or newer is recommended. Other distributions work if the packages below are available.

2.3 System packages

The build needs a C/C++ toolchain, OpenCV development headers, libclang (for generating the OpenCV Rust bindings) and libudev (for serial-port discovery):

```
sudo apt update && sudo apt install -y \  
    clang libclang-dev llvm-dev \  
    libudev-dev libopencv-dev pkg-config
```

Package	Why it is needed
clang, libclang-dev, llvm-dev	The <code>opencv</code> crate generates Rust bindings from the C++ headers at build time using libclang.
libopencv-dev	OpenCV itself (video capture, image processing).
libudev-dev	Required by the <code>serialport</code> crate for USB/serial device discovery.
pkg-config	Lets the build scripts locate the installed libraries.

Table 2.1: System package roles

2.4 Rust toolchain

Install the stable Rust toolchain via `rustup`:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

2.5 Serial port permissions

On Linux the serial devices belong to the `dialout` group:

```
sudo usermod -a -G dialout $USER
# Log out and back in for the change to take effect.
```

2.6 Building and first run

```
cd deltax2sort
cargo build           # compile (also builds the Slint UI)
cargo test           # run the unit test suite
cargo run -- --mock   # start with simulated hardware
cargo run             # start against the real hardware
```

Tip

Always bring the system up in mock mode first (`cargo run - --mock`) after installing or changing the configuration. Mock mode exercises the full control flow — UI, orchestrator, queueing, E-stop — without moving any hardware.

2.7 Troubleshooting

clang-sys cannot find libclang

Install `libclang-dev`. If only a versioned library such as `libclang-21.so.1` exists, create a symlink named `libclang.so` (the repository keeps one under `libs/`) and point the build at it:

```
ln -sf /usr/lib/x86_64-linux-gnu/libclang-21.so.1 libs/libclang.so
LIBCLANG_PATH=$PWD/libs cargo build
```


libudev-sys build failure

libudev-dev is missing; install it and rebuild.

OpenCV build failure

Ensure libopencv-dev and pkg-config are installed and that `pkg-config --modversion opencv4` prints a version.

Permission denied opening /dev/ttyUSB*

Your user is not in the `dialout` group yet, or you have not logged out and back in.

Robot fails the IsDelta handshake

Check the port assignment (robot vs. conveyor may be swapped), the baud rate, and that no other program holds the port open. See [Appendix A](#) for the handshake protocol.

3 Configuration

3.1 The configuration file

All settings live in a single TOML file, `Settings.toml` in the working directory by default; an alternative path can be given with `-config <path>`. If the file does not exist, a default one is created on first start.

Note

The configuration is **validated at startup**. Inconsistent values — for example a `z_pick` below the robot's physical range, or a drop position outside the workspace — abort startup with a message naming the offending key. The application never starts with a configuration that could command the robot outside its workspace.

3.2 Section `[robot]`

Key	Default	Meaning
<code>port_name</code>	<code>/dev/ttyUSB0</code>	Serial device of the robot controller.
<code>baud_rate</code>	<code>115200</code>	Serial speed (Delta X2 standard).
<code>home_on_connect</code>	<code>true</code>	Home (G28) automatically after connecting.
<code>x_min, x_max</code>	<code>-160, 160</code>	Allowed X range in mm.
<code>y_min, y_max</code>	<code>-160, 160</code>	Allowed Y range in mm.
<code>z_min, z_max</code>	<code>-200, 0</code>	Allowed Z range in mm (Z= 0 is the top).
<code>z_pick</code>	<code>-180</code>	Z height at which bricks are picked (belt surface).
<code>z_travel</code>	<code>-50</code>	Safe Z height for horizontal travel.
<code>feed_rate</code>	<code>15000</code>	G-code feed rate in mm/min (F parameter).

Table 3.1: `[robot]` keys

Constraints enforced by validation: each axis minimum must be below its maximum; `z_pick` and `z_travel` must lie within `[z_min, z_max]`; `z_travel` must be above `z_pick`.

Safety warning

Reducing `x_min/x_max/y_min/y_max` below the physical maximum is a legitimate way to keep the robot away from fixtures near the belt. Never *extend* them beyond the Delta X2 specification (X/Y ± 160 mm, Z $[-200, 0]$ mm; see Appendix A).

3.3 Section `[conveyor]`

3.4 Section `[camera]`

The camera model is not fixed yet, so all capture parameters are configurable rather than hard-coded. The requested values are best-effort: the device may pick the nearest mode it supports,

Key	Default	Meaning
port_name	/dev/ttyUSB1	Serial device of the belt controller.
baud_rate	115200	Serial speed.
default_speed	1000	Raw S value sent with M3 on start.
speed_mm_s	100.0	Belt speed in robot coordinates, mm/s. Signed: positive means objects travel toward +Y. Used by the trajectory planner.

Table 3.2: [conveyor] keys

Key	Default	Meaning
device_id	0	OpenCV device index (/dev/video0 → 0).
width, height	1280, 720	Requested capture resolution.
fps	30	Requested capture frame rate.
fourcc	(unset)	Optional pixel-format FOURCC (e.g. "MJPG"); many UVC cameras need it to reach full frame rate at higher resolutions. Unset keeps the driver default.

Table 3.3: [camera] keys

and the driver logs (and adopts) the resolution and frame rate actually in effect at connection time. Calibration is always based on the effective resolution.

3.5 Section [sorting]

Key	Default	Meaning
drop_x, drop_y, drop_z	150, 0, -100	Position of the drop bin. Must be inside the robot workspace.

Table 3.4: [sorting] keys

3.6 Section [vision]

3.7 Camera calibration

The pixel-to-robot transform is affine: scale (mm per pixel), rotation and offset. There is deliberately no built-in default: parameters are constructed from the capture resolution actually in effect (the helper `CalibrationParams::centered(width, height, mm_per_px)` maps the optical centre to the robot origin). A guided calibration wizard is planned (see the TODO list); until it exists, the parameters in `src/vision/calibration.rs` must be adjusted to match the physical camera mounting.

3.8 Complete example

```
[robot]
port_name = "/dev/ttyUSB0"
baud_rate = 115200
home_on_connect = true
```

Key	Default	Meaning
threshold	60.0	Fixed binary threshold (0–255) applied after grayscale conversion and blur.
min_area, max_area	500, 10000	Contour area window in pixels ² for a blob to count as an object.
invert	true	true : objects darker than the belt; false : objects lighter than the belt.

Table 3.5: [vision] keys

```

x_min = -150.0
x_max = 150.0
y_min = -150.0
y_max = 150.0
z_min = -200.0
z_max = 0.0
z_pick = -180.0
z_travel = -150.0
feed_rate = 15000

[conveyor]
port_name = "/dev/ttyUSB1"
baud_rate = 115200
default_speed = 1000
speed_mm_s = 100.0

[camera]
device_id = 0
width = 1280
height = 720

[sorting]
drop_x = 150.0
drop_y = 0.0
drop_z = -100.0

[vision]
threshold = 60.0
min_area = 500.0
max_area = 10000.0
invert = true

```


4 Operation

4.1 Command line

Option	Effect
-m, -mock	Use simulated robot, conveyor and camera. No hardware required.
-c, -config <path>	Configuration file (default <code>Settings.toml</code>).
-h, -help	Show help.

Table 4.1: Command-line options

Logging uses `env_logger`; set `RUST_LOG` to control verbosity:

```
RUST_LOG=info cargo run -- --mock      # normal operation
RUST_LOG=debug cargo run                # includes G-code level tracing
```

4.2 Startup sequence

On start the application: loads and validates the configuration; connects to the robot (`IsDelta` handshake, Appendix A) and homes it if `home_on_connect` is set; connects to the conveyor; acquires the emergency-stop handles; connects to the camera; starts the orchestrator; and finally opens the operator UI. Any failure in this chain aborts startup with a message identifying the failing device.

4.3 Operator interface

Control	Behaviour
Start Sorting / Pause System	Toggles the conveyor. Starts the belt at <code>default_speed</code> ; pressing again stops it. The label reflects the current state.
Stop Conveyor	Stops the belt (enabled while running).
Home Robot	Queues a homing command through the orchestrator (enabled while the system is not running).
EMERGENCY STOP	Immediately halts robot and conveyor and clears all queued work. See section 4.4.

Table 4.2: UI controls

The dashboard shows the robot connection status, the conveyor state and the total number of sorted bricks. The live camera feed and detection overlays are not wired up yet (see the TODO list).

4.4 Emergency stop and recovery

Pressing **EMERGENCY STOP**:

1. writes M112 to the robot and M5 to the conveyor on dedicated serial handles, bypassing every queue and lock;
2. clears the orchestrator's instruction queue and pauses it;
3. marks both devices as E-STOP in the UI.

To recover after an E-stop:

1. Remove the physical cause (jam, obstruction, misplaced fixture).
2. Restart the application. The robot controller may also need a power cycle after M112, depending on firmware.
3. Home the robot before resuming production.

Note

If a robot command fails during operation (timeout, serial error, robot fault), the orchestrator clears its queue and pauses itself automatically rather than continuing with a desynchronised state. The incident is logged with the failing command.

4.5 Routine operation checklist

1. Check that the belt is empty and the workspace is clear.
2. Start the application; confirm *Status: Connected*.
3. If not homed automatically, press *Home Robot*.
4. Press *Start Sorting*.
5. Supervise the first picks; keep a hand near the E-stop.

5 Maintenance and Development

5.1 Project layout

Path	Contents
<code>src/main.rs</code>	Entry point, UI callbacks, task wiring.
<code>src/hardware.rs</code>	Robot, conveyor and camera drivers plus mocks; workspace limits; emergency-stop handles.
<code>src/orchestrator.rs</code>	Message loop, instruction queue, trajectory planner.
<code>src/vision/</code>	Detector, calibration, tracker, classifier.
<code>src/app_config.rs</code>	Configuration structures, loading, validation.
<code>ui/app_window.slint</code>	Operator interface definition.
<code>docs/</code>	Specifications, this manual (sources under <code>docs/manual/</code>), TODO list.

Table 5.1: Repository layout

5.2 Tests

Run the unit test suite with `cargo test`. Covered today: configuration parsing and validation (including backward compatibility with older configuration files), the pixel-to-world calibration transform, the trajectory-planner intercept mathematics, workspace-limit enforcement in both robot drivers, and the instruction queue.

5.3 Development workflow

Develop against mock hardware (`-mock`) whenever possible. The mocks implement the same traits as the real drivers, including workspace-limit enforcement, so orchestration logic can be validated safely. The robot’s serial protocol is documented in [Appendix A](#).

5.4 Editing this manual

The manual sources live under `docs/manual/`, one file per chapter; `docs/manual.tex` is the driver that sets the class and includes them. Build with:

```
cd docs && latexmk -pdf manual.tex
```

5.5 Remaining work

The authoritative, maintained list of open work items lives in `docs/TODO.md`. Major open areas at the time of writing: wiring the vision loop end-to-end (camera → detection → pick scheduling), the live camera feed and overlays in the UI, a real object tracker and classifier, belt-motion compensation using robot position feedback, and a camera calibration wizard.

A Delta X2 G-code Protocol Reference

This appendix is the authoritative description of the G-code dialect and communication protocol of the Delta X2 robot as used by this project.

A.1 Communication protocol

Parameter	Value
Transport	USB serial (CDC), typically <code>/dev/ttyUSB*</code> or <code>/dev/ttyACM*</code>
Baud rate	115 200
Line ending	<code>\n</code> (newline)
Handshaking	The robot answers commands; for critical synchronisation use the <code>FEEDBACK</code> parameter (section A.4).

Table A.1: Serial link parameters

A.2 Identification and status

A.2.1 IsDelta

Queries the device to confirm it is a Delta X robot.

- **Response:** `YesDelta`

The application performs this handshake on every connection and refuses to proceed if the answer does not arrive.

A.2.2 DeltaState

Queries the current operating state of the robot.

Response	Meaning
<code>Free</code>	Idle and ready.
<code>Running</code>	Executing a command.
<code>Wait</code>	Waiting for input or synchronisation.
<code>Almostdone</code>	Near the end of a movement.
<code>Done</code>	Movement or command completed.

Table A.2: DeltaState responses

A.2.3 Position

Returns the current coordinates of the robot.

- **Response format:** `X:<val> Y:<val> Z:<val> W:<val> U:<val>`

Note

Position is not used by the application yet; it is the basis for the planned belt-motion compensation (robot position feedback for the trajectory planner — see `docs/TODO.md`).

A.3 Movement commands

A.3.1 G00 / G01 — linear movement

Moves the end-effector to the specified coordinates.

- **Syntax:** `G01 X<val> Y<val> Z<val> F<speed>`
- **F** is the feed rate in mm/min (configured via `robot.feed_rate`).

```
G01 X10.5 Y-20.0 Z-150.0 F2000
```

A.3.2 G28 — homing

Moves all axes to their home position (top endstops).

- **Origin (X0 Y0 Z0):** after homing, the robot is positioned at the top centre.
- **Z axis:** moves downward (negative values). $Z=0$ is the top; $Z=-200$ is near the bottom of the workspace.

A.3.3 G90 / G91 — positioning mode

- **G90:** interpret subsequent coordinates as absolute positions relative to the origin. The application sets this mode right after homing.
- **G91:** interpret subsequent coordinates as displacements from the current position (useful for jogging).

A.4 Synchronisation — FEEDBACK

FEEDBACK:<string> can be appended to any G-code command. The robot echoes **<string>** back on the serial port once the command has been *physically executed* — not merely received.

```
G01 X0 Y0 Z-50 FEEDBACK:done
```

Response (after the move finishes):

```
done
```

Note

The robot driver appends a unique **FEEDBACK:sync_<n>** id to every command and blocks until the echo returns. This is what makes a completed `move_to` call mean “the arm has arrived”, and it is why every command has an overall timeout: a lost echo surfaces as an error instead of a hang.

Command	Effect
M03	Output on (open gripper / start vacuum).
M05	Output off.
M112	Emergency stop (used by the E-stop path).

Table A.3: M-codes

A.5 End effector controls

A.6 Physical workspace (SP-X2)

Axis	Range
X	−160 mm to +160 mm
Y	−160 mm to +160 mm
Z	−200 mm to 0 mm (0 at the top)
Working diameter	320 mm

Table A.4: Workspace of the SP-X2 model

These are the hard physical limits; the configured workspace ([`robot`] section, chapter on configuration) must always be equal to or smaller than them.

A.7 Quick reference

Command	Meaning
IsDelta	Handshake; the robot answers <code>YesDelta</code> .
DeltaState	Operating state query.
Position	Current coordinates query.
G28	Home all axes (to the top; X0 Y0 Z0 afterwards).
G90 / G91	Absolute / relative positioning mode.
G01 X.. Y.. Z.. F..	Linear move at feed rate F (mm/min).
M03 / M05	End effector (gripper/pump) on / off.
M112	Emergency stop.
FEEDBACK:<id>	Echoed back when the command has physically completed.

Table A.5: Protocol summary

Index

- architecture, 1
- belt speed, 7
- building, 4
- calibration, 8
- camera, 1, 7
 - calibration, 8
- cargo, 4
- cargo test, 13
- checklist, 12
- classifier, 2
- command line, 11
- configuration, 7
 - [camera], 7
 - [conveyor], 7
 - [robot], 7
 - [sorting], 8
 - [vision], 8
- controls, 11
- conveyor, 1, 7
- coordinate transform, 8
- dashboard, 11
- Delta X2 robot, 1
- detection, 2, 8
- development workflow, 13
- dialout group, 4
- drop bin, 8
- emergency stop, 2, 11, 17
- feed rate, 7, 16
- G-code, 15
 - DeltaState, 15
 - FEEDBACK, 16
 - G01, 16
 - G28, 16
 - G90, 16
 - G91, 16
 - IsDelta, 15
 - M03, 17
 - M05, 17
 - M112, 17
 - Position, 15
- gripper, 1, 17
- handshake, 11, 15
- hardware
 - components, 1
 - layer, 2
- hardware requirements, 3
- homing, 11, 16
- installation, 3
- instruction queue, 2
- latexmk, 13
- libclang, 3
- LIBCLANG_PATH, 4
- libudev, 3
- logging, 11
- M112, 11
- manual
 - editing, 13
- mock mode, 2, 4, 11, 13
- movement, 16
- OpenCV, 3
- operation, 11
- orchestrator, 2
- pick height, 7
- position feedback, 15
- prerequisites, 3
- project layout, 13
- protocol
 - serial, 15
- Raspberry Pi, 3
- recovery, 11
- roadmap, 13
- Rust
 - toolchain, 4
- RUST_LOG, 11
- rustup, 4
- safety, 2
- serial port
 - permissions, 4
- Settings.toml, 7

- Slint, [1](#)
- sorting, [1](#)
- SP-X2, [17](#)
- startup sequence, [11](#)
- synchronisation, [16](#)
- system packages, [3](#)
- tests, [13](#)
- threshold, [8](#)
- TODO list, [13](#)
- tokio, [1](#)
- touch display, [3](#)
- tracker, [2](#)
- troubleshooting, [4](#)
- user interface, [11](#)
- validation
 - configuration, [2](#), [7](#)
- vision, [2](#)
- workspace
 - limits, [2](#), [7](#)
 - physical limits, [17](#)